PROJECT REPORT

Automated Business Intelligence & Dashboard Refresh Pipeline

Submitted by: **Aditya Mishra**

BCA Student, Lovely Professional University (Expected 2027)

Date: 6 November 2026

GitHub Repository: https://github.com/Adi-dot0/automated-bi-etl-pipeline

1. Abstract

This project is an automated ETL (Extract, Transform, Load) pipeline in Python that prepares business data for Business Intelligence reporting. It extracts sales, customer and product data, cleans and transforms it, loads it into a star-schema analytical warehouse, calculates sales, customer and product KPIs, checks alert rules such as a revenue drop of more than 20 percent, and exports fresh Power BI-ready tables on every run. On the sample dataset provided with the project, the pipeline processed 63 raw sales rows, removed 2 duplicates and 2 invalid rows, loaded 59 clean orders worth Rs 1,74,500 in total revenue, and correctly raised a revenue-drop alert for October 2026.

2. Business Problem

Businesses collect data in many places: Excel spreadsheets, CRM systems, databases and sales platforms. Analysts manually combine these sources, clean them, update reports and refresh dashboards. This creates delays, inconsistent reporting and repeated manual effort. A centralized automated pipeline removes this repeated work and makes the same clean, updated data available for every dashboard refresh.

3. Objectives

- Extract business data from file sources, with a ready pattern for API sources
- Transform data: cleaning, revenue calculation, customer segmentation and a date dimension
- Load data into a star-schema warehouse (one fact table and dimension tables)
- Calculate sales, customer and product KPIs automatically on every run
- Raise alerts for important changes such as a revenue drop of more than 20 percent
- Log every pipeline run so failures can be traced
- Export refreshed tables that a Power BI dashboard can connect to

4. Technology Stack

Tool	Use in this project
Python	ETL automation
Pandas	Data extraction and transformation
SQL	Warehouse schema and KPI queries (database/schema.sql, queries.sql)
SQLite	Local analytical warehouse (same schema also runs on PostgreSQL)
Power BI	Dashboard layer - connects to the pipeline outputs (guide in power_bi folder)
Apache Airflow	Optional scheduling - DAG template provided in dags folder
Requests	API extraction pattern (extraction/api_extractor.py)
GitHub	Version control and project hosting

5. System Architecture

Data Sources (CSV / API) → Python Extraction → Data Validation and Cleaning → Transformation (Python + SQL) → Star-Schema Warehouse → KPI Calculation → Alerts + Power BI-Ready Exports → Power BI Dashboard

6. Modules of the System

Module 1 - Data Extraction (extraction/)

file_extractor.py reads the sales, customer and product CSV files and supports date-based filtering for incremental runs. api_extractor.py provides the API pattern from the specification: authorization header, pagination and error handling, ready to connect to a real CRM or product API.

Module 2 - Data Transformation (transformation/)

cleaning.py removes duplicate orders and drops invalid rows (missing or negative quantity, invalid dates), converts data types and calculates revenue as quantity multiplied by unit price. business_metrics.py builds the date dimension (year, month, weekday), segments customers as New, Regular or Premium based on total spend, and calculates monthly KPIs, product KPIs and headline KPIs.

Module 3 - Database Loading (loading/database_loader.py)

Creates the warehouse from database/schema.sql and loads the star schema: fact_sales (order_id, customer_id, product_id, order_date, quantity, revenue) with dim_customer, dim_product and dim_date dimension tables. Every run is also recorded in etl_execution_log with start and end time, records processed and status.

Module 4 - KPI Monitoring (transformation/business_metrics.py + database/queries.sql)

Sales KPIs: total revenue, total orders, average order value, monthly revenue growth. Customer KPIs: total and repeat customers, customer segments. Product KPIs: top products by revenue and revenue share. The same KPIs are also written as SQL queries in database/queries.sql for use inside the warehouse or a BI tool.

Module 5 - Automated Alerts (monitoring/alerts.py)

Rules: revenue drop of more than 20 percent compared to the previous month; a high-value customer whose total spend crosses the threshold in config; monthly revenue below a minimum threshold; and pipeline failure, which is recorded in the execution log and the pipeline log file. Alerts are written to reports/alerts.txt on every run.

Module 6 - Dashboard Refresh (reports/ exports + power_bi guide)

Every pipeline run refreshes the warehouse and the export files (fact_sales, dim_customer, KPI tables). Power BI Desktop connects to these outputs, so a dashboard built on them shows new data after each run. Scheduled refresh in Power BI Service depends on the dataset type, licence

and gateway requirements, as noted in the specification; the guide in the power_bi folder explains both the file connection and the database connection options.

7. How to Run the Project

```
pip install -r requirements.txt
python main.py
```

The pipeline prints the headline KPIs, monthly KPIs, top products and any alerts, writes the warehouse file data_warehouse.db, and refreshes all report files in the reports folder.

8. Testing and Results

Unit tests

Six unit tests in the tests folder cover sales cleaning (duplicates and invalid rows), the date dimension, customer segmentation, monthly KPI growth calculation, headline KPIs and the revenue-drop alert rule. All 6 tests pass.

Results on sample data

The sample data (sample_data folder) contains 12 customers, 8 products and 63 sales rows for August to October 2026, with 4 problem rows added on purpose: 2 duplicate orders, 1 row with a missing quantity and 1 row with a negative quantity. Verified results:

Measure	Result
Sales rows extracted	63
Duplicates removed	2
Invalid rows removed	2
Rows loaded in fact_sales	59
Total revenue	Rs 1,74,500
Total orders	59
Average order value	Rs 2,957.63
Repeat customers	11 out of 12
Customer segments	Premium 8, Regular 3, New 1
Top product	Mechanical Keyboard - Rs 59,976 (34.4% of revenue)

Monthly revenue: August Rs 49,311, September Rs 83,509 (growth +69.4 percent), October Rs 41,680 (growth -50.1 percent). Because October revenue fell by more than 20 percent, the revenue-drop alert fired correctly, and 5 high-value customer alerts were raised for customers with total spend of Rs 15,000 or more. The etl_execution_log recorded the run as Success with 59 records processed.

9. Limitations

- The warehouse runs on SQLite locally so the project works on any machine without a server; the same schema is provided for PostgreSQL for a team setup

- A Power BI .pbix file is not included because it must be created in Power BI Desktop; the project provides the refreshed data and a step-by-step connection guide instead

- The API extractor is a ready template; the demonstration pipeline runs on the local sample files, not a live API

- The Airflow DAG is a template for when Airflow is installed; scheduling also works with cron or Windows Task Scheduler

- Alerts are written to the log and a text file; sending email alerts needs the user's own SMTP settings

10. Future Scope

Connect a real data source through the API extractor, move the warehouse to PostgreSQL, build the Power BI dashboard from the provided guide and publish it with scheduled refresh, and add the third portfolio project (Automated Web Scraping and CRM Synchronization with HubSpot).

11. Conclusion

The pipeline automates the complete path from raw business files to dashboard-ready data: extraction, cleaning, transformation, star-schema loading, KPI calculation, alert checking and run logging happen in a single command. Testing on sample data with known problems showed correct cleaning (59 of 63 rows loaded), correct KPI values verified both in Python and with SQL queries on the warehouse, and correct alert behaviour. The complete project is available on GitHub at https://github.com/Adi-dot0/automated-bi-etl-pipeline

Appendix - Project Files

main.py; extraction/file_extractor.py and api_extractor.py; transformation/cleaning.py and business_metrics.py; loading/database_loader.py; database/schema.sql and queries.sql; monitoring/etl_logger.py and alerts.py; config/config.json; dags/etl_dag.py; power_bi/POWER_BI_GUIDE.md; sample_data (3 CSV files); reports (KPI tables, exports, alerts); tests/test_transformation.py and test_metrics.py; requirements.txt; README.md.